

Projeto 1 - Análise espectral por transformadas de Fourier.

2 de abril de 2025

Universidade de São Paulo - Instituto de Física de São Carlos.

Docente: Francisco C. Alcaraz

Tony Maciel Alexander. # USP: 12624850

Tarefa 1

Faça um programa que calcule de forma direta (sem qualquer método de aceleração ou otimização) transformadas de Fourier discreta, assim como transformada inversa, para uma série temporal de N dados obtidos em intervalo temporal Δt . A série temporal a ser analisada será dada em um arquivo "data.in" e a resultante colocada em "data.out".

A convenção usada para a transformada de Fourier discreta (DFT) será dada pela equação (1) :

$$y_j = \frac{1}{N} \sum_{k=0}^{N-1} Y_k e^{-2\pi i j k / N} \iff Y_k = \sum_{j=0}^{N-1} y_j e^{2\pi i j k / N} \quad (1)$$

onde y_j é um sinal medido nos tempos $\Delta t, 2\Delta t, \dots, N\Delta t$ e Y_k é a transformada de Fourier desse sinal.

Para resolver essa tarefa, decidi isolar as funções que calculam o DFT e sua inversa num módulo chamado "Fourier_Transform.f90" (vide fig 1)¹². Fiz isso para facilitar a escrita das demais tarefas, uma vez que todos requerem essas funções. Além disso, serve também para facilitar a leitura dos códigos, pois cada módulo nesse projeto tem somente uma funcionalidade.

¹Vale a pena dizer que tentei criar um código universal que sempre forneceria os mesmos resultados, independentemente do compilador usado. Por isso usei as definições de precisão de variáveis reais do módulo intrínseco "iso_fortran_env". Porém usei a função "zexp()" que não faz parte dos padrões de Fortran (são específicos do compilador GNU. Então se outro compilador for usado, meus códigos provavelmente não vão compilar). Fiz isso para obter precisão dupla nos meus resultados. Para ter um código mais universal, eu poderia usar a função "cexp()" que fornece precisão simples.

²Para usar esse módulo nos programas, basta compilá-lo com a opção de compilação `-c -o Fourier_transform.o`, depois compilar o programa em questão com `"[programa].f90 Fourier_transform.o"`.

```

12 module Fourier_Transform
13     ! cexp -> simple precision , zexp -> double precision (not fortran standard)
14     use, intrinsic :: iso_fortran_env, only: sp=>real32 , dp=>real64
15     implicit none
16
17     integer, parameter      :: dpc = kind((1.0_dp, 1.0_dp)) ! for double precision complex numbers
18     real(dp), parameter    :: twopi = 8.0_dp*atan(1.0_dp)
19     complex(dpc), parameter :: im = (0.0_dp , 1.0_dp)
20
21     contains
22
23     subroutine FT(Y, res, N)
24         ! Calculates FT of Y and puts result in res(:)
25         complex(dpc) , intent(in)      :: Y(:) ! Signal separated by EQUAL time intervals.
26         complex(dpc) , intent(in out) :: res(:) ! FT of Y.
27         integer , intent(in)           :: N      ! length of Y and res.
28         integer                        :: j, k
29
30         do k = 0, N-1 ! for each FT component...
31             res(k+1) = (0.0_dp , 0.0_dp)
32             do j = 0, N-1 ! get FT from Y...
33                 res(k+1) = res(k+1) + Y(j+1)*zexp(twopi * im * j * k /real(N,dp))
34             end do
35         end do
36
37     end subroutine FT
38
39
40     subroutine IFT(Y, res, N)
41         ! Calculates inverse FT of Y and puts result in res(:)
42         complex(dpc) , intent(in)      :: Y(:) ! Signal separated by EQUAL time intervals.
43         complex(dpc) , intent(in out) :: res(:) ! Inverse FT of Y.
44         integer , intent(in)           :: N      ! length of Y and res.
45         integer                        :: j, k
46
47         do j = 0, N-1 ! for each FT component...
48             res(j+1) = (0.0_dp , 0.0_dp)
49             do k = 0, N-1 ! get IFT from Y...
50                 res(j+1) = res(j+1) + Y(k+1)*zexp(-twopi * im * j * k / real(N,dp))
51             end do
52             res(j+1) = res(j+1) / N
53         end do
54
55     end subroutine IFT
56 end module Fourier_Transform

```

NORMAL F Fourier_Transform.f90

Figura 1: Código fonte para calcular o DFT.

Com esse módulo, usei-lo no programa final "tarefa-1-12624850.f90" simbolicamente, pois a tarefa não pede pra rodar algo (vide fig 2). Nesse programa final, eu só escrevi como que eu faria para registrar um sinal num arquivo de entrada "data.in", depois como ler um sinal num arquivo e por fim, como escrever o DFT desse sinal num arquivo de saída "data.out" (chamando a função do módulo "Fourier_Transform.f90").

```

18 use Fourier_Transform
19 implicit none
20
21 real(dp), parameter :: pi = 4.0_dp*atan(1.0_dp)
22 real(dp), parameter :: dt = 1e-2_dp ! time interval
23 integer, parameter :: N = 1000 ! number of time intervals
24 real(dp), parameter :: a1 = 2.0_dp, a2 = 4.0_dp, w1 = 4.0_dp * pi, w2 = 2.5_dp * pi ! w = 2pif
25 complex(dp) :: Y(N) , FTY(N)
26 real(dp) :: read_dt
27 integer :: read_N
28 integer :: i, io1, io2, io3
29
30 ! Initialize data to file
31 open(newunit=io1,file="data.in",action="write")
32 write(io1,*) N, dt
33 do i = 1, N
34 | !Y(i) = a1*dcos(w1*i*dt) + a2*dsin(w2*i*dt)
35 | write(io1,*) cmplx(a1*dcos(w1*i*dt) + a2*dsin(w2*i*dt), kind=dp)
36 end do
37 close(io1)
38
39 ! Read data from file. first line has N and dt, rest is the signal from times dt to N*dt
40 open(newunit=io2,file="data.in",action="read")
41 read(io2,*) read_N , read_dt
42 do i = 1, N
43 | read(io2,*) Y(i)
44 end do
45 close(io2)
46
47 ! Get Fourier transform
48 call FT(Y, FTY, read_N)
49
50 ! Save data to file (magnitude plot)
51 open(newunit=io3,file="data.out",action="write")
52 do i = 1, read_N
53 | write(io3,*) (i-1)/(read_dt*real(read_N,dp)) , abs(FTY(i))
54 end do
55 close(io3)
56
57 end

```

NORMAL F tarefa-1-12624850.f90

Figura 2: Código simbólico para usar o módulo do DFT.

Dentro de "data.in" tem: N e Δt na primeira linha, seguido pelos N valores do sinal ao longo do tempo.

Dentro de "data.out" tem por linha: frequência e o valor absoluto do DFT do sinal (pois o DFT retorna números complexos). A frequência por linha é:

$$f = \frac{i}{N\Delta t} \quad , \quad i = 0, 1, \dots, N$$

Tarefa 2

Teste seu programa gerando as séries:

$$y_i = a_1 \cos(\omega_1 t_i) + a_2 \sin(\omega_2 t_i), \quad t_i = i\Delta t, \quad i = 1, \dots, N. \quad (2)$$

Escolha:

- (a) $N = 200$, $\Delta t = 0.04$, $a_1 = 2$, $a_2 = 4$, $\omega_1 = 4\pi\text{Hz}$, $\omega_2 = 2.5\pi\text{Hz}$
- (b) $N = 200$, $\Delta t = 0.04$, $a_1 = 3$, $a_2 = 2$, $\omega_1 = 4\pi\text{Hz}$, $\omega_2 = 2.5\pi\text{Hz}$
- (c) $N = 200$, $\Delta t = 0.4$, $a_1 = 2$, $a_2 = 4$, $\omega_1 = 4\pi\text{Hz}$, $\omega_2 = 0.2\pi\text{Hz}$
- (d) $N = 200$, $\Delta t = 0.4$, $a_1 = 3$, $a_2 = 2$, $\omega_1 = 4\pi\text{Hz}$, $\omega_2 = 0.2\pi\text{Hz}$

Compare graficamente e discuta as diferenças, se houver, entre as séries: (a) e (b); (c) e (d); (a) e (c); (b) e (d).

O código fonte dessa tarefa é essencialmente o mesmo da tarefa anterior, vide fig 3 (Só foi mudado os nomes dos arquivos de escrita e leitura para cada item para facilitar a organização).

Em geral, é esperado que os gráficos da magnitude do DFT em função da frequência sejam nulos em todo ponto, mas tenham picos nas frequências que correspondem às frequências que compõem o sinal original. (as frequências dos picos ocorrerão em $f = \omega_n/2\pi$)

```

18 use Fourier_Transform
19 implicit none
20
21 real(dp), parameter :: pi = 4.0_dp*atan(1.0_dp)
22 real(dp), parameter :: dt = 4e-1_dp ! time interval
23 integer, parameter :: N = 200 ! number of time intervals
24 real(dp), parameter :: a1 = 3.0_dp, a2 = 2.0_dp, w1 = 4.0_dp * pi, w2 = 0.2_dp * pi ! w = 2pif
25 complex(dpc) :: Y(N) , FTY(N)
26 real(dp) :: read_dt
27 integer :: read_N
28 integer :: i, io1, io2, io3
29
30 ! Initialize data to file
31 open(newunit=io1,file="entrada-2-4-12624850.in",action="write")
32 write(io1,*) N, dt
33 do i = 1, N
34 | !Y(i) = a1*dcos(w1*i*dt) + a2*dsin(w2*i*dt)
35 | write(io1,*) cmplx(a1*dcos(w1*i*dt) + a2*dsin(w2*i*dt), kind=dp)
36 end do
37 close(io1)
38
39 ! Read data from file. first line has N and dt, rest is the signal from times dt to N*dt
40 open(newunit=io2,file="entrada-2-4-12624850.in",action="read")
41 read(io2,*) read_N , read_dt
42 do i = 1, N
43 | read(io2,*) Y(i)
44 end do
45 close(io2)
46
47 ! Get Fourier transform
48 call FT(Y, FTY, read_N)
49
50 ! Save data to file (magnitude plot)
51 open(newunit=io3,file="saida-2-4-12624850.out",action="write")
52 do i = 1, read_N
53 | write(io3,*) (i-1)/(read_dt*real(read_N,dp)) , abs(FTY(i))
54 end do
55 close(io3)
56
57 end

```

NORMAL F tarefa-2-12624850.f90

Figura 3: Programa que calcula o DFT de várias séries.

Antes de mostrar os gráficos, vale a pena notar que pelo fato do sinal ser real, a magnitude do seu DFT em função da frequência será simétrica em torno da frequência Nyquist. Isso ocorre porque pela equação (1), para um sinal real:

$$Y_{(N-k)} \triangleq \sum_{j=0}^{N-1} y_j e^{2\pi i j (N-k)/N} = \sum_{j=0}^{N-1} y_j e^{2\pi i j} e^{-2\pi i j k/N} = \sum_{j=0}^{N-1} y_j (e^{2\pi i j k/N})^* = \left(\sum_{j=0}^{N-1} y_j e^{2\pi i j k/N} \right)^*$$

$$\therefore Y_{(N-k)} = Y_k^* = Y_k$$

Logo, há simetria em torno da frequência no meio que é justamente a frequência Nyquist. Por isso que os gráficos a seguir apresentarão o dobro de picos esperados. Para ilustrar isso, vide o grafico 4 da série (a).

Magnitude plot of DFT of real signal.

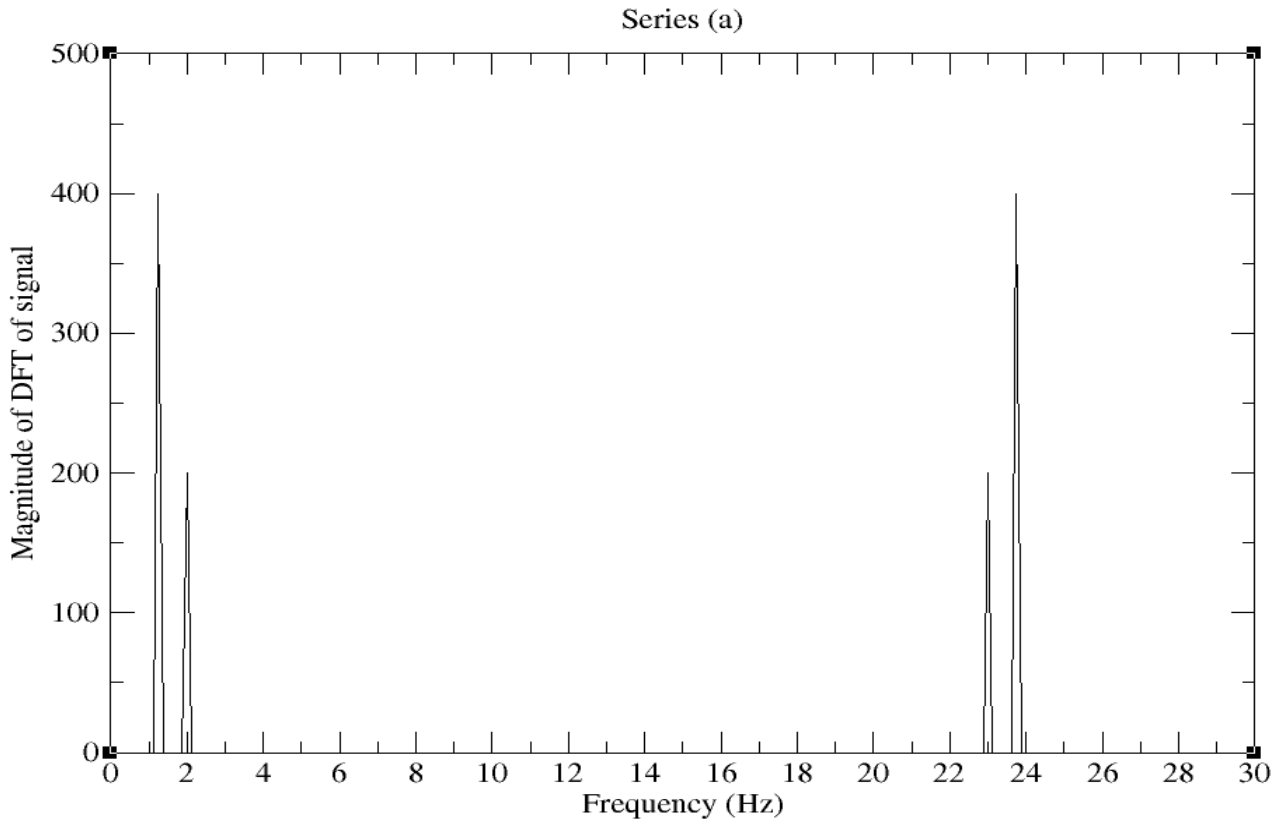


Figura 4: série (a). Picos simétricos sobre frequência Nyquist.

A seguir, nos gráficos 5 e 6, estão plotados as séries (a) e (b) , e (c) e (d). As diferenças entre essas séries são as mesmas: a magnitude do DFT tem picos de tamanhos diferentes. Isso ocorre porque pela equação (1), é como se cada termo da soma foi multiplicado por um número, então o seu módulo varia proporcionalmente a esse número (e isso ocorre separadamente para cada pico em questão, porque a soma pode ser separada). Intuitivamente, o tamanho do pico mostra a "relevância" daquela frequência para o sinal original.

Magnitude plot of series (a) and (b)

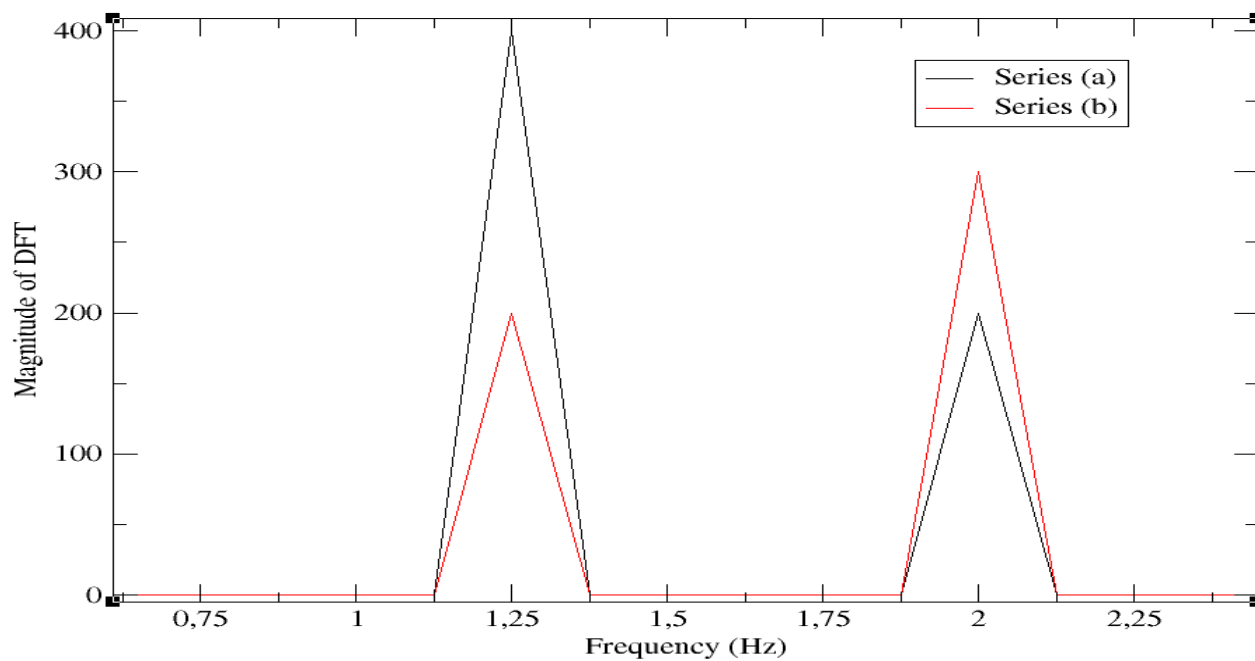


Figura 5: série (a) e (b).

Magnitude plot of series (c) and (d)

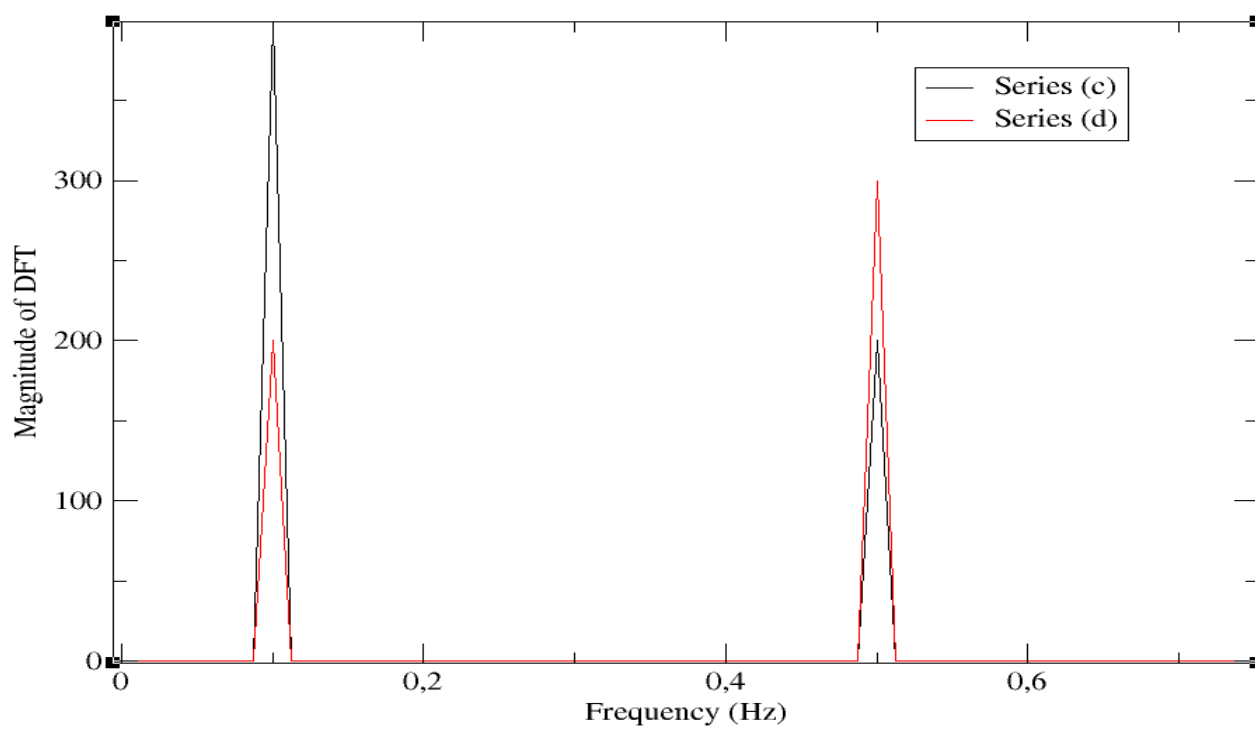


Figura 6: série (c) e (d).

A figura 5 está de acordo com o esperado, mas os picos da figura 6 parecem estar nos lugares errados. Na verdade isso ocorre por causa da escolha de dt frente às frequências envolvidas no sinal original: para um sinal composto por certas frequências, uma escolha de dt acarreta no registro desse sinal a cada dt segundos. Mas e se o sinal tiver uma frequência bem aguda, tal que a escolha de dt seja muito grande? Então nesse caso, perdemos informação sobre o sinal.

No caso das séries (c) e (d), $dt = 0.4$ s, o que significa que o sinal original é registrado com uma frequência de $f_s = 1/dt = 2.5$ Hz ("sampling frequency"). Então, por definição, a frequência Nyquist será $f_{nyq} = f_s/2 = 1.25$ Hz. Mas perceba que o sinal das séries (c) e (d) têm frequências de $4\pi/2\pi = 2$ Hz e $0.2\pi/2\pi = 0.1$ Hz. Como foi mostrado que os gráficos são simétricos com respeito à frequência Nyquist, a frequência de 0.1 Hz aparecerá como um dos dois primeiros picos (porque é menor que a frequência Nyquist), enquanto a frequência de 2 Hz aparecerá com um dos dois últimos picos (vide figura 7). Mas e os demais picos em 0.5 Hz e 2.4 Hz? Esses são por causa de "aliasing": Eu registro um sinal (de 2 Hz) com frequência 2.5 Hz, então devo ficar com impressão que registrei um sinal de $2.5 - 2.0 = 0.5$ Hz. Analogamente para a outra frequência do sinal, devo pensar que também registrei um sinal de $2.5 - 0.1 = 2.4$ Hz.

Magnitude plot of series (c) and (d)

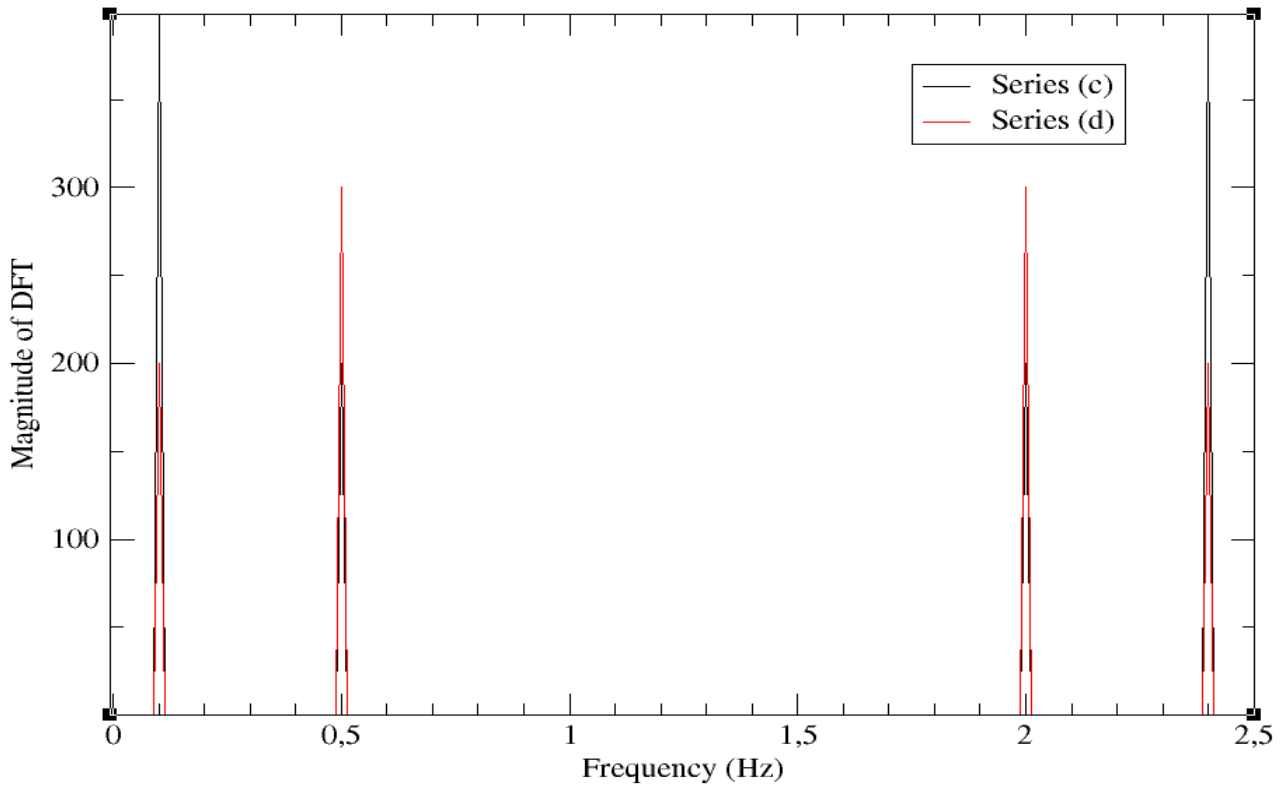


Figura 7: série (c) e (d) completa. Frequência Nyquist é 1.25 Hz: equidistante dos dois picos centrais.

Comparando agora as séries (a) e (c), temos os gráficos 8 e 9.

Para a série (a), temos: $f_s = 1/0.04 = 25$ Hz, $f_{nyq} = f_s/2 = 12.5$ Hz e frequências do sinal iguais a $4\pi/2\pi = 2$ Hz e $2.5\pi/2\pi = 1.25$ Hz. Então é esperado que os seus picos estejam no intervalo de 0 a 25 Hz, simétricos em relação a 12.5 Hz e quatro picos centrados em: 1.25 Hz, 2 Hz, $25 - 2 = 23$ Hz e $25 - 1.25 = 23.75$ Hz (pelos mesmos motivos explicados anteriormente). Como $a_1 = 2$ e $a_2 = 4$,

é esperado que os picos em ω_1 (2 Hz e 23 Hz) tenham tamanhos menores que aqueles em ω_2 (1.25 Hz e 23.75 Hz).

Para a série (c), temos: $f_s = 1/0.4 = 2.5$ Hz, $f_{nyq} = f_s/2 = 1.25$ Hz e frequências do sinal iguais a $4\pi/2\pi = 2$ Hz e $0.2\pi/2\pi = 0.1$ Hz. Então é esperado que os seus picos estejam no intervalo de 0 a 2.5 Hz, simétricos em relação a 1.25 Hz e quatro picos centrados em: 0.1 Hz, 2 Hz, $2.5 - 2 = 0.5$ Hz e $2.5 - 0.1 = 2.4$ Hz. Como $a_1 = 2$ e $a_2 = 4$, é esperado que os picos em ω_1 (2 Hz e 0.5 Hz) tenham tamanhos menores que aqueles em ω_2 (0.1 Hz e 2.4 Hz).

Magnitude plot of series (a) and (c)

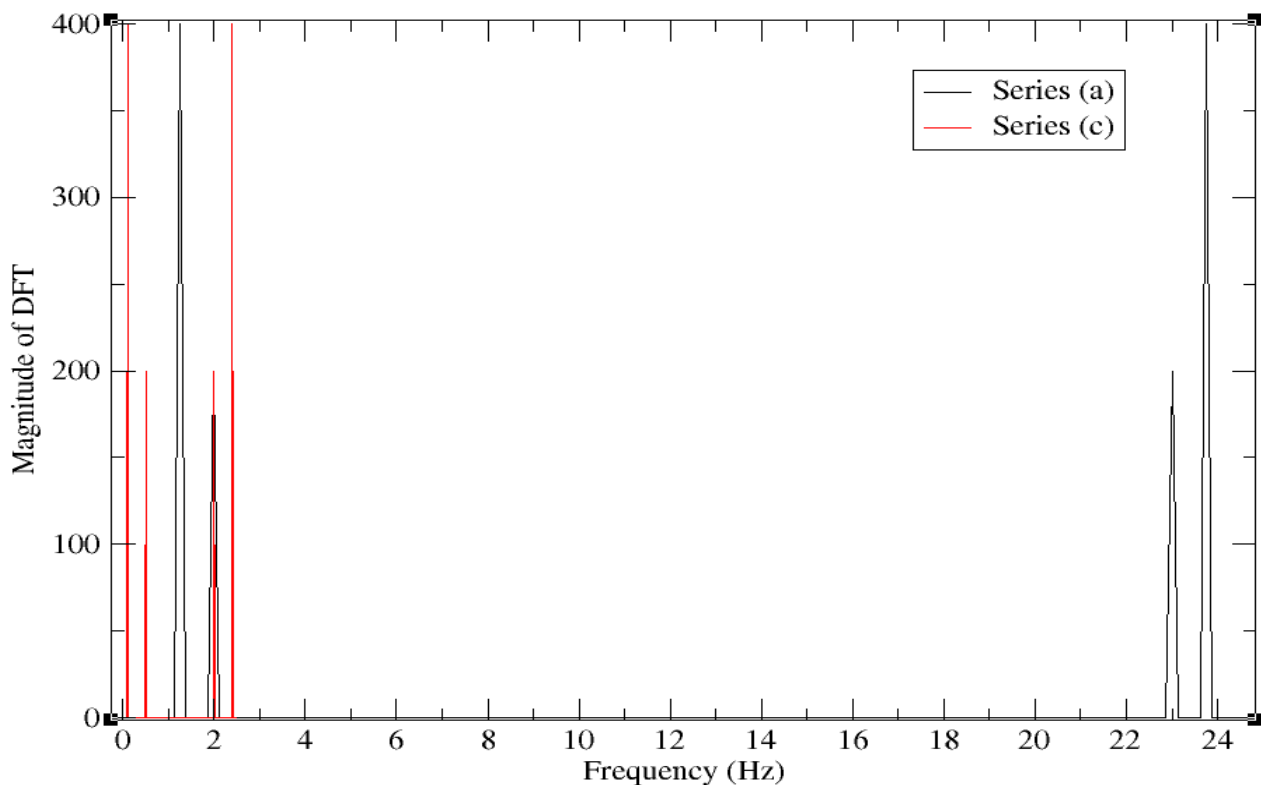


Figura 8: série (a) e (c) completa.

Magnitude plot of series (a) and (c)

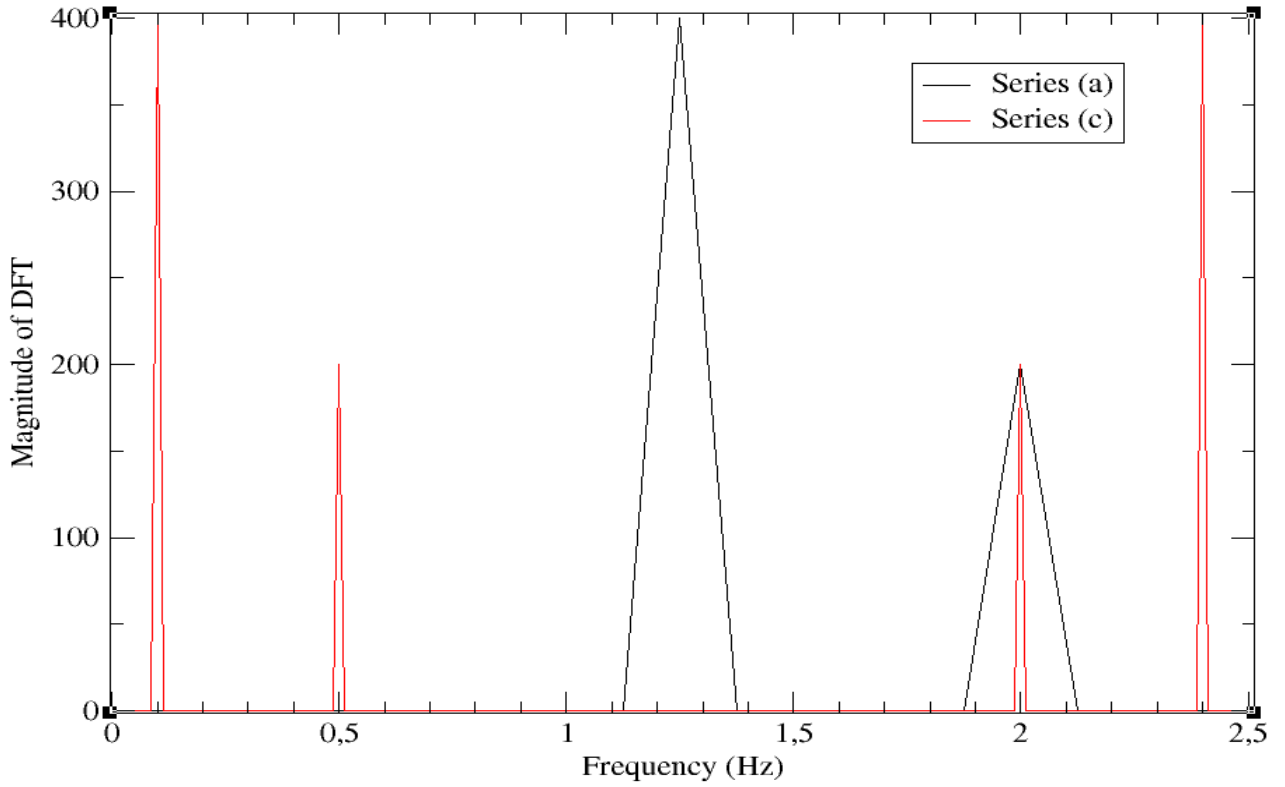


Figura 9: série (a) e (c) com zoom.

Comparando agora as séries (b) e (d), temos os gráficos 10 e 11.

Para a série (b), temos: $f_s = 1/0.04 = 25$ Hz, $f_{nyq} = f_s/2 = 12.5$ Hz e frequências do sinal iguais a $4\pi/2\pi = 2$ Hz e $2.5\pi/2\pi = 1.25$ Hz. Então é esperado que os seus picos estejam no intervalo de 0 a 25 Hz, simétricos em relação a 12.5 Hz e quatro picos centrados em: 1.25 Hz, 2 Hz, $25 - 2 = 23$ Hz e $25 - 1.25 = 23.75$ Hz. Como $a_1 = 3$ e $a_2 = 2$, é esperado que os picos em ω_1 (2 Hz e 23 Hz) tenham tamanhos maiores que aqueles em ω_2 (1.25 Hz e 23.75 Hz).

Para a série (d), temos: $f_s = 1/0.4 = 2.5$ Hz, $f_{nyq} = f_s/2 = 1.25$ Hz e frequências do sinal iguais a $4\pi/2\pi = 2$ Hz e $0.2\pi/2\pi = 0.1$ Hz. Então é esperado que os seus picos estejam no intervalo de 0 a 2.5 Hz, simétricos em relação a 1.25 Hz e quatro picos centrados em: 0.1 Hz, 2 Hz, $2.5 - 2 = 0.5$ Hz e $2.5 - 0.1 = 2.4$ Hz. Como $a_1 = 3$ e $a_2 = 2$, é esperado que os picos em ω_1 (2 Hz e 0.5 Hz) tenham tamanhos maiores que aqueles em ω_2 (0.1 Hz e 2.4 Hz).

Magnitude plot of series (b) and (d)

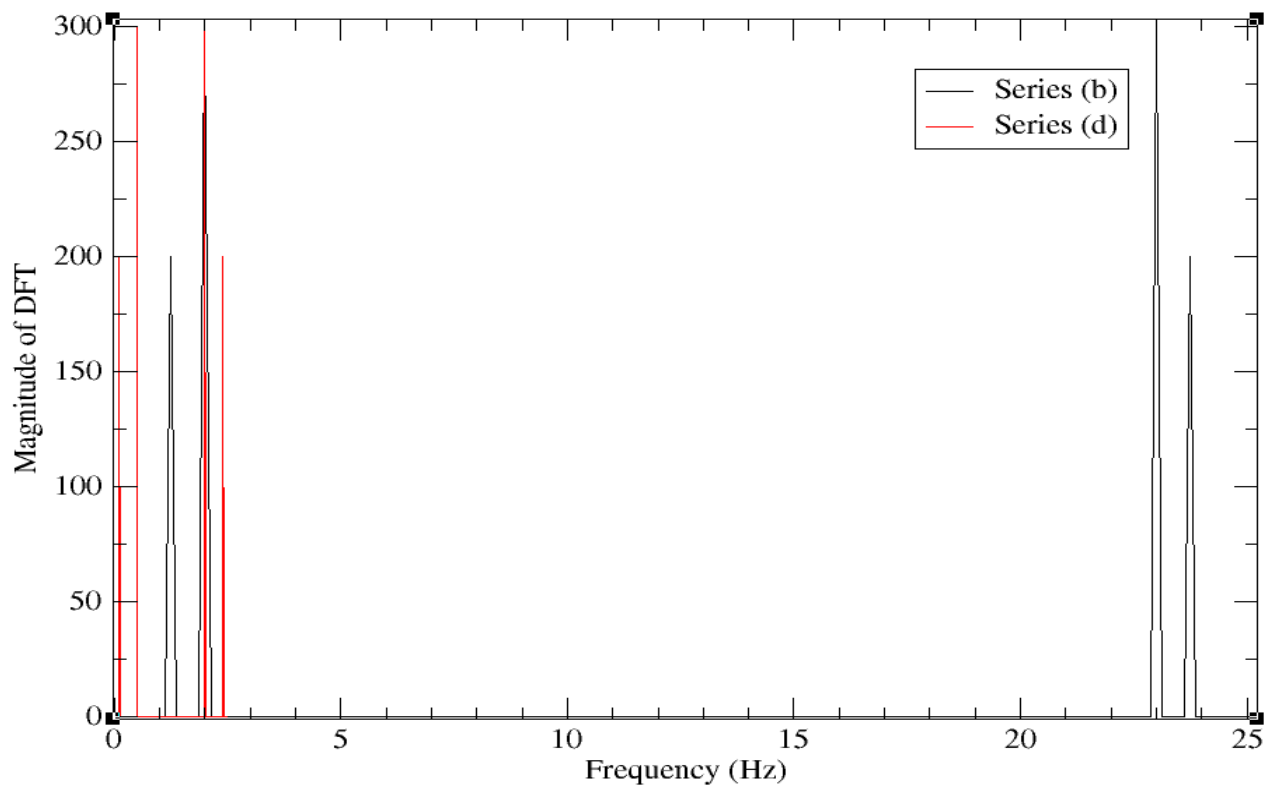


Figura 10: série (b) e (d) completa.

Magnitude plot of series (b) and (d)

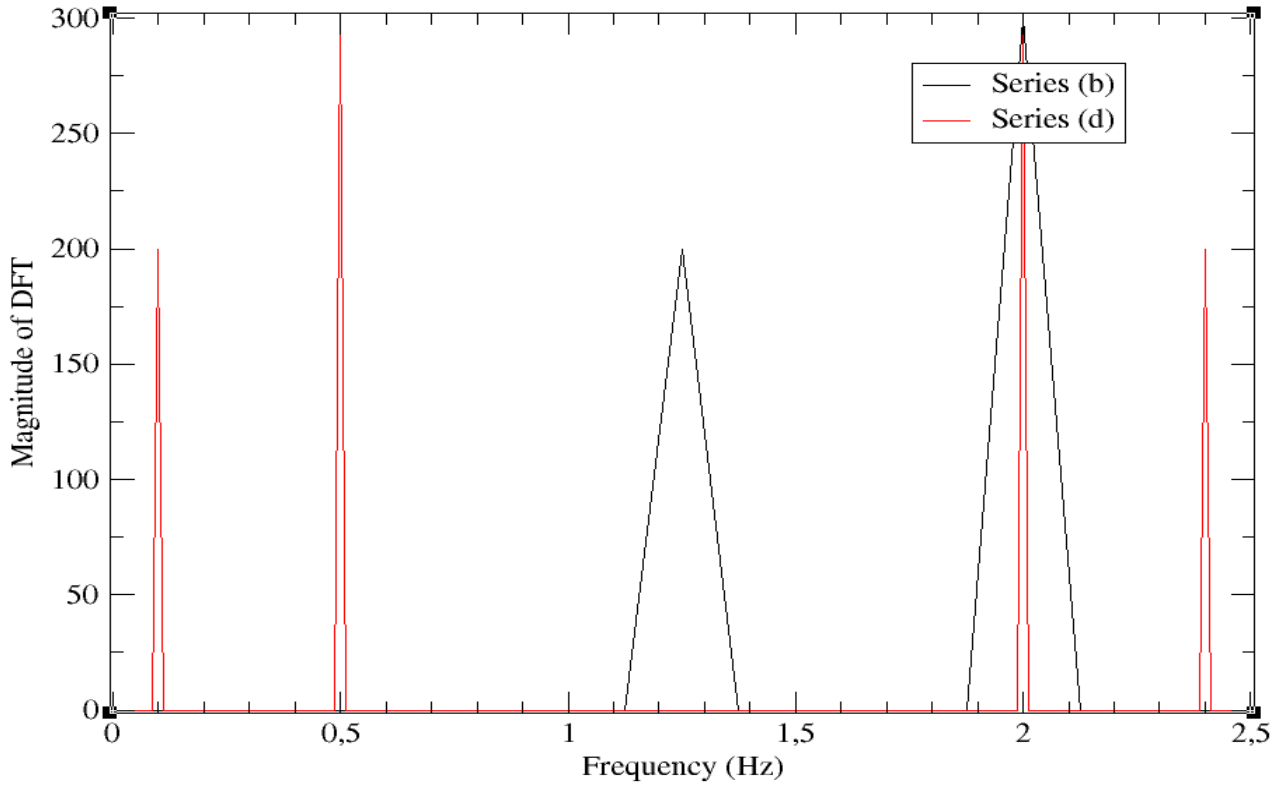


Figura 11: série (b) e (d) com zoom.

Tarefa 3

Escolha agora na expressão (2) as séries:

(e) $N = 200$, $\Delta t = 0.04$, $a_1 = 2$, $a_2 = 4$, $\omega_1 = 4\pi\text{Hz}$, $\omega_2 = 1.4\pi\text{Hz}$

(f) $N = 200$, $\Delta t = 0.04$, $a_1 = 2$, $a_2 = 4$, $\omega_1 = 4.2\pi\text{Hz}$, $\omega_2 = 1.4\pi\text{Hz}$

Compare o comportamento espectral dos dois gráficos obtidos com aqueles das séries (a) e (c), discuta as diferenças.

O código fonte dessa tarefa é idêntica com o da tarefa anterior (fig 3), então vou omití-la.

Para a série (e), temos: $f_s = 1/0.04 = 25\text{ Hz}$, $f_{nyq} = f_s/2 = 12.5\text{ Hz}$ e frequências do sinal iguais a $4\pi/2\pi = 2\text{ Hz}$ e $1.4\pi/2\pi = 0.7\text{ Hz}$. Então é esperado que os seus picos estejam no intervalo de 0 a 25 Hz, simétricos em relação a 12.5 Hz e quatro picos centrados em: 0.7 Hz, 2 Hz, $25 - 2 = 23\text{ Hz}$ e $25 - 0.7 = 24.3\text{ Hz}$. Como $a_1 = 2$ e $a_2 = 4$, é esperado que os picos em ω_1 (2 Hz e 23 Hz) tenham tamanhos menores que aqueles em ω_2 (0.7 Hz e 24.3 Hz).

Para a série (f), temos: $f_s = 1/0.04 = 25\text{ Hz}$, $f_{nyq} = f_s/2 = 12.5\text{ Hz}$ e frequências do sinal iguais a $4.2\pi/2\pi = 2.1\text{ Hz}$ e $1.4\pi/2\pi = 0.7\text{ Hz}$. Então é esperado que os seus picos estejam no intervalo de 0 a 25 Hz, simétricos em relação a 12.5 Hz e quatro picos centrados em: 0.7 Hz, 2.1 Hz, $25 - 2.1 = 22.9\text{ Hz}$ e $25 - 0.7 = 24.3\text{ Hz}$. Como $a_1 = 2$ e $a_2 = 4$, é esperado que os picos em ω_1 (2.1 Hz e 22.9 Hz) tenham tamanhos menores que aqueles em ω_2 (0.7 Hz e 24.3 Hz).

Magnitude plot of series (e) and (f)

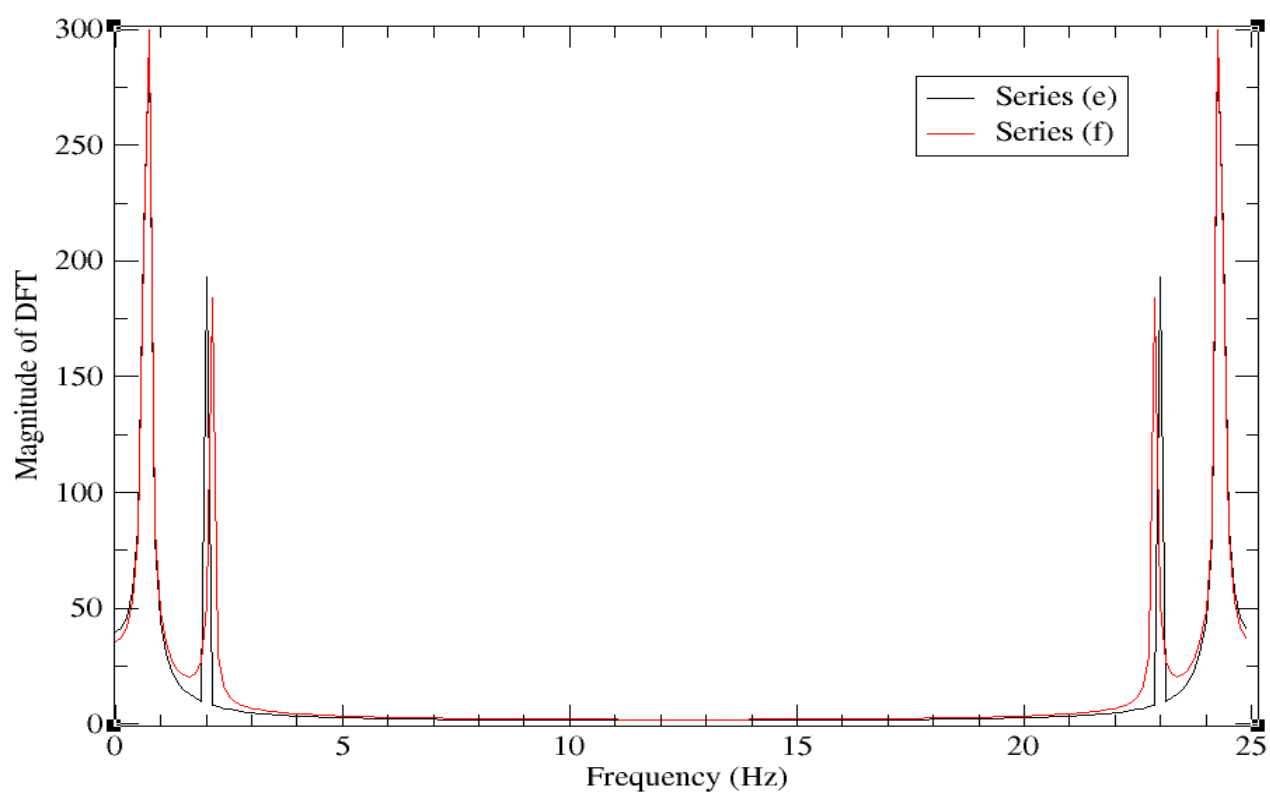


Figura 12: série (e) e (f) completa.

Magnitude plot of series (e) and (f)

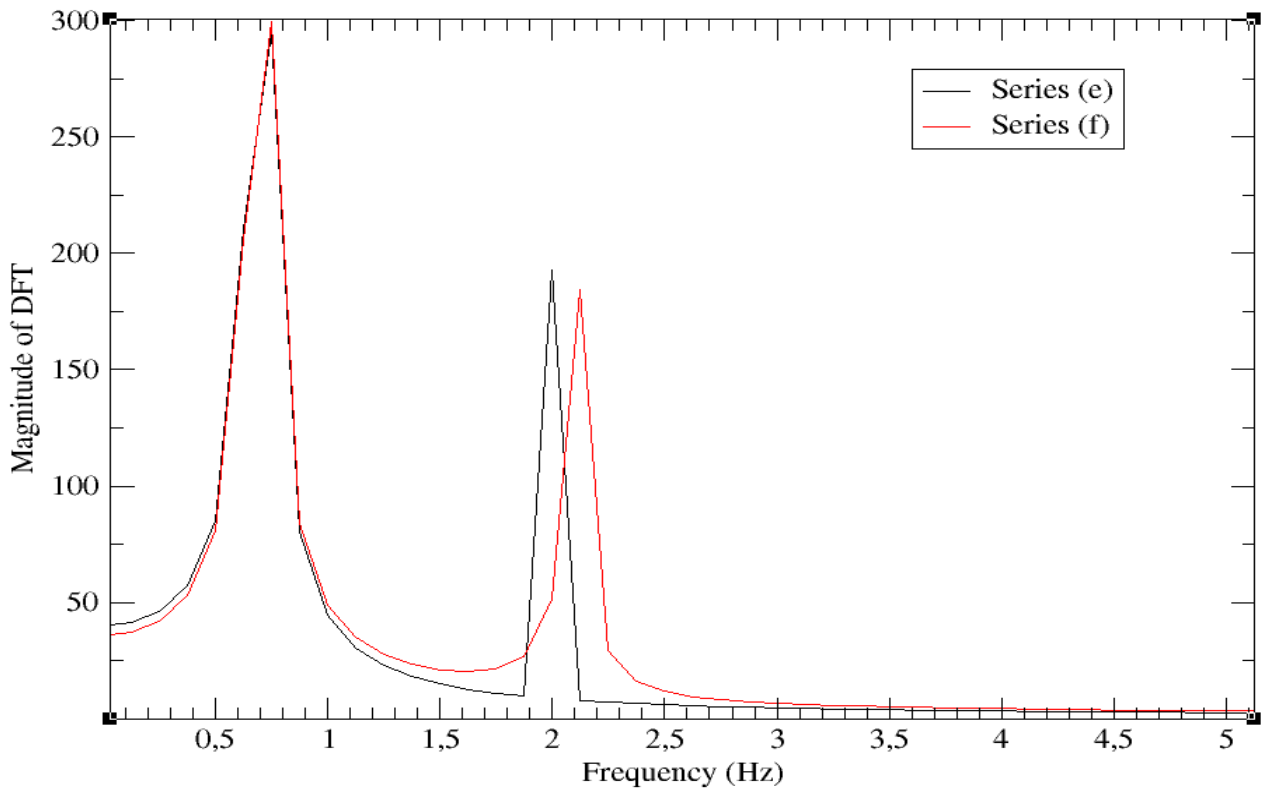


Figura 13: série (e) e (f) com zoom.

Em comparação com as séries (a) e (c), as séries (e) e (f) aparentam ter curvas mais suaves e picos menos abruptos. Isso provavelmente ocorre por conta da frequência $\omega_2 = 1.4\pi$ Hz que não tem nas séries (a) e (c). Isto é, o espaçamento mínimo entre as frequências nas séries (a), (e) e (f) é $df = 1/Ndt = 1/(200 \times 0.04) = 0.125$ Hz, enquanto na série (c) isso é $df = 1/(200 \times 0.4) = 0.0125$ Hz. Então, como as frequências presentes nas séries (a) e (c) são múltiplos dos seus df , temos que os gráficos das suas magnitudes poderão apresentar exatamente as frequências em questão. Enquanto para as séries (e) e (f), como 0.7 Hz não é múltiplo de 0.125 Hz, essa frequência em particular cairá entre as frequências $5 \times 0.125 = 0.625$ Hz e $6 \times 0.125 = 0.75$ Hz, o que dá a impressão de picos menos suaves. Incidentalmente, como a série (f) tem a frequência 2.1 Hz também, que não é múltiplo exato de $df = 0.125$ Hz (fica entre $df \times 16 = 2$ Hz e $df \times 17 = 2.125$ Hz), então essa série terá duas frequências que não podem ser representadas exatamente, o que provavelmente dá a impressão de curvas mais suaves em comparação com a série (e), que só tem uma frequência desse tipo.

Tarefa 4

Tome os resultados provenientes da série (a) ("data.out") e o utilize como dado de entrada para o cálculo da transformada inversa de Fourier. Compare a série obtida com a série (a) inicialmente usada. Discuta.

Para essa tarefa, eu essencialmente repeti o programa da tarefa 2 (a), mas depois li o arquivo de saída, calculei a transformada inversa de Fourier e salvei a parte real do sinal devolvido num outro arquivo de saída com o sinal original (vide fig 14 para o código fonte e fig 15 para o resultado)³.

³Para a visualização chamei `xmgrace` com o comando `-nxy` que plota toda outra coluna em função da primeira

Além disso, como o sinal devolvido pelo inverso do DFT é complexo (por padrão), eu quis ver se o sinal era realmente real (que nem o sinal original). Pra fazer isso, calculei o maior valor em módulo da parte imaginária do sinal devolvido pela função inversa e obtive um resultado da ordem de 10^{-13} , que é essencialmente negligenciável.

```

25 real(dp)                :: read_dt, temporary, max_im ! maximum imaginary part is ~1e-13
26 integer                 :: i, io1, io2, io3, io4, io5, read_N
27
28 ! Initialize data to file
29 open(newunit=io1,file="entrada-4-1-12624850.in",action="write")
30   write(io1,*) N, dt
31   do i = 1, N
32     !Y(i) = a1*dcos(w1*i*dt) + a2*dsin(w2*i*dt)
33     write(io1,*) cplx(a1*dcos(w1*i*dt) + a2*dsin(w2*i*dt), kind=dp)
34   end do
35 close(io1)
36
37 ! Read data from file. first line has N and dt, rest is the signal from times dt to N*dt
38 open(newunit=io2,file="entrada-4-1-12624850.in",action="read")
39   read(io2,*) read_N , read_dt
40   do i = 1, N
41     read(io2,*) Y(i)
42   end do
43 close(io2)
44
45 ! Get Fourier transform
46 call FT(Y, FTY, read_N)
47
48 ! Save data to file
49 open(newunit=io3,file="saida-4-1-12624850.out",action="write")
50   do i = 1, read_N
51     write(io3,*) (i-1)/(read_dt*real(read_N,dp)) , FTY(i) ! not using abs() to recover signal
52   end do
53 close(io3)
54
55 ! read output of FT for IFT
56 open(newunit=io4,file="saida-4-1-12624850.out",action="read")
57   do i = 1, read_N ! must be same amount of frequencies as times.
58     read(io4,*) temporary , IFTY(i) !
59   end do
60 close(io3)
61
62 ! Get Inverse Fourier transform
63 call IFT(FTY, IFTY, read_N)
64
65 ! Save data to file (only plotting real part because I know the signal is real)
66 max_im = 0.0_dp
67 open(newunit=io5,file="saida-4-2-12624850.out",action="write")
68   do i = 1, read_N
69     write(io5,*) i*read_dt , IFTY(i)%re, a1*dcos(w1*i*dt) + a2*dsin(w2*i*dt)
70     if (max_im .lt. abs(IFTY(i)%im)) max_im = IFTY(i)%im
71   end do
72   print*, max_im
73 close(io5)
74 end

```

NORMAL F tarefa-4-12624850.f90

Figura 14: Código fonte para ver se a inversa do DFT coincide com o sinal original.

coluna

Inverse DFT compared with original signal

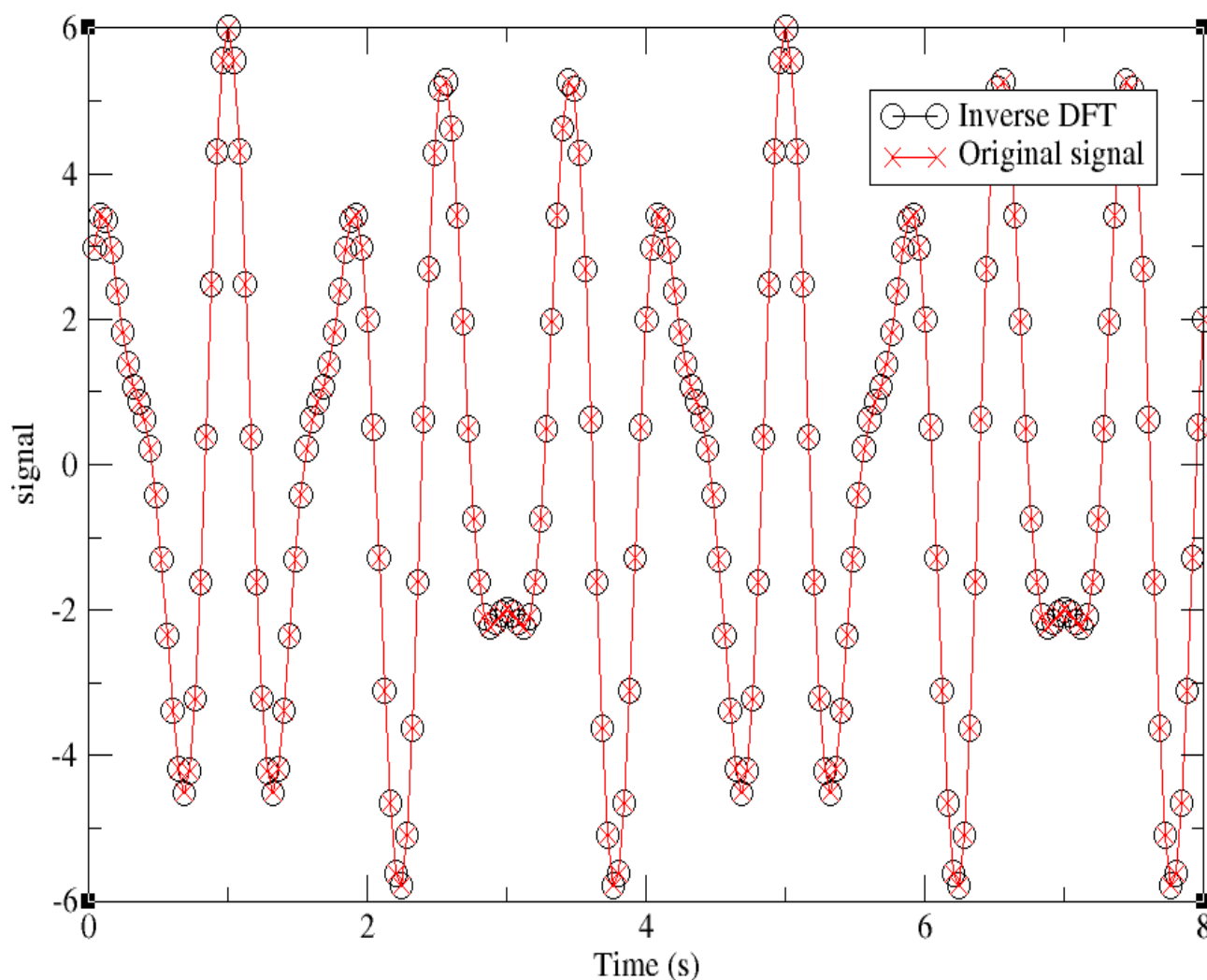


Figura 15: Inverso do DFT de um sinal e este sinal. Praticamente não há diferença entre os sinais.

Pelo gráfico anterior, não é possível ver diferença apreciável entre o sinal original e a inversa do DFT.

Tarefa 5

Use os parâmetros da série (a) para $N = 50, 100, 200$ e 400 e mostre que o tempo de cálculo computacional cresce com N^2 .

em princípio é fácil ver que a complexidade de tempo é quadrática, porque o DFT (pela definição) precisa de dois laços de tamanho N para ser computada. Então a quantidade de operações a serem realizadas vai com N^2 .

Computacionalmente, para mostrar que a complexidade de tempo do DFT vai com $\mathcal{O}(N^2)$, optei por um método diferente. Note que se o tempo de execução depender do quadrado do número de dados, então:

$$t = N^2 \implies \log(t) = 2 \log(N) \quad (3)$$

Logo, se eu efetuar o DFT para vários valores de N e medir o tempo médio de execução para cada um, depois plotar $\log(t)$ versus $\log(N)$, devo obter uma reta com coeficiente angular igual a 2 (vide fig 16 para o código fonte e fig 17 para o resultado)⁴⁵.

```

12 use Fourier_Transform
13 implicit none
14
15 real(dp), parameter :: pi = 4.0_dp*atan(1.0_dp)
16 real(dp), parameter :: dt = 4e-2_dp ! time interval
17 integer, parameter :: N_max = 400 ! max number of time intervals
18 real(dp), parameter :: a1 = 2.0_dp, a2 = 4.0_dp, w1 = 4.0_dp * pi, w2 = 2.5_dp * pi ! w = 2pif
19 complex(dpc) :: Y(N_max) , FTY(N_max)
20 real(dp) :: start, fin, average
21 integer :: i, io, N, total
22
23 ! Save data to file
24 open(newunit=io,file="saida-5-12624850.out",action="write")
25 total = 50 ! number of averages for times
26 do N = 50, N_max
27     do i = 1, N
28         Y(i) = cmplx(a1*dcos(w1*i*dt) + a2*dsin(w2*i*dt), kind=dp)
29     end do
30
31     ! time FT
32     average = 0.0_dp
33     do i = 1, total
34         call cpu_time(start)
35         call FT(Y(1:N), FTY(1:N), N)
36         call cpu_time(fin)
37         average = average + (fin - start)
38     end do
39
40     write(io,*) dlog(real(N,dp)), dlog(average/total)
41 end do
42 close(io)
43 end

```

Figura 16: Código fonte para calcular o tempo de execução do DFT em função do número de dados.

⁴O coeficiente angular foi calculado pela função intrínseca do xmgrace de regressão linear.

⁵Para essa figura, percebi que o coeficiente angular era mais próximo de 2 se eu explicitamente compilar com a opção `-O0`, que proíbe quaisquer otimizações. Se eu compilar com `-O3` ou `-Ofast`, há mais flutuações entre os tempos e o ajuste do xmgrace fica pior.

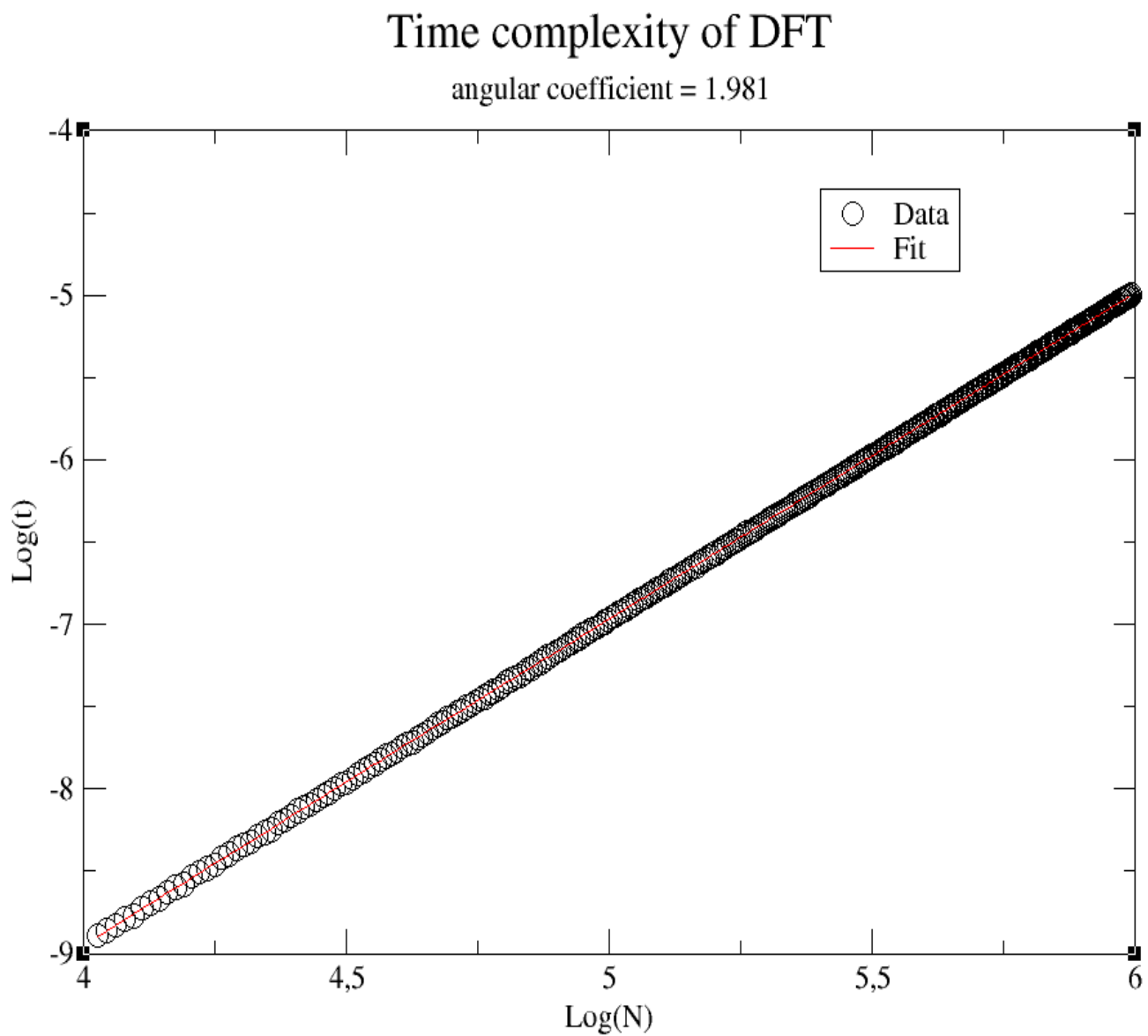


Figura 17: Tempo de execução do DFT em função do número de dados. O coeficiente angular é aproximadamente 2.

Referência

- [1] - Nicholas J. Giordano, Computational Physics (Prentice Hall, New Jersey, 1997).